

# CHAPTER 1

## INTRODUCTION

MICROBLOGGING sites such as Twitter have reshaped the way people find, share, and disseminate timely information. Many organizations have been reported to create and monitor targeted Twitter streams to collect and understand users' opinions. Targeted Twitter stream is usually constructed by filtering tweets with predefined selection criteria (e.g., tweets published by users from a geographical region, tweets that match one or more predefined keywords).

Due to its invaluable business value of timely information from these tweets, it is imperative to understand tweets' language for a large body of downstream applications, such as named entity recognition (NER) event detection and summarization opinion mining sentiment analysis, and many others. Given the limited length of a tweet (i.e., 140 characters) and no restrictions on its writing styles, tweets often contain grammatical errors, misspellings, and informal abbreviations.

The error-prone and short nature of tweets often make the word-level language models for tweets less reliable. For example, given a tweet "I call her, no answer. Her phone in the bag, she dancin," there is no clue to guess its true theme by disregarding word order (i.e., bag-of-word model). The situation is further exacerbated with the limited context provided by the tweet. That is, more than one explanation for this tweet could be derived by different readers if the tweet is considered in isolation. On the other hand, despite the noisy nature of tweets, the core semantic information is well preserved in tweets in the form of named entities or semantic phrases. For example, the emerging phrase "she dancin" in the related tweets indicates that it is a key concept—it classifies this tweet into the family of tweets talking about the song "She Dancin", a trend topic in Bay Area in January 2013.

In this paper, we focus on the task of tweet segmentation. The goal of this task is to split a tweet into a sequence of consecutive  $n$ -grams ( $n \geq 1$ ), each of which is called a segment. A segment can be a named entity (e.g., a movie title "finding nemo"), a semantically meaningful information unit (e.g., "officially released"), or any other types of phrases which appear "more than by chance" Fig. 1 gives an example. In this example, a tweet "They said to spare no effort to increase traffic throughput on circle line." is split into eight segments. Semantically meaningful

segments “spare no effort”, “traffic throughput” and “circle line” are preserved. Because these segments preserve semantic meaning of the tweet more precisely than each of its constituent words does, the topic of this tweet can be better captured in the subsequent processing of this tweet. For instance, this segment-based representation could be used to enhance the extraction of geographical location from tweets because of the segment “circle line”.

## 1.2 Objective Of The Introduction

In fact, segment-based representation has shown its effectiveness over word-based representation in the tasks of named entity recognition and event detection

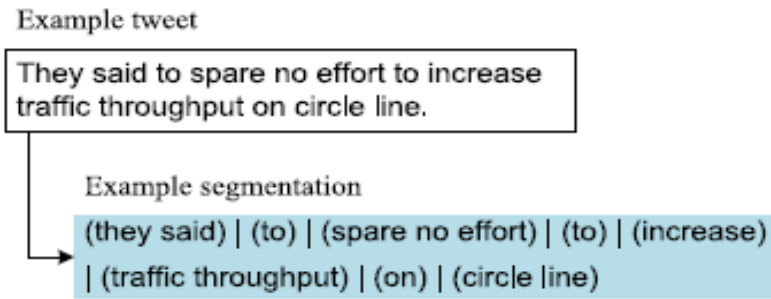


Fig. 1. Example of tweet segmentation.

Note that, a named entity is valid segment; but a segment may not necessarily be a named entity. In the segment “korea versus greece” is detected for the event related to the world cup match

between Korea and Greece. To achieve high quality tweet segmentation, we propose a generic tweet segmentation framework, named HybridSeg. HybridSeg learns from both global and local contexts, and has the ability of learning from pseudo feedback. Global context. Tweets are posted for information sharing and communication. The named entities and semantic phrases are well preserved in tweets. The global context derived from Web pages (e.g., Microsoft Web N-Gram corpus) or Wikipedia therefore helps identifying the meaningful segments in tweets. The method realizing the proposed framework that solely relies on global context is denoted by HybridSegWeb. Local context. Tweets are highly time-sensitive so that many emerging phrases

## **CHAPTER 2**

### **LITERATURE SURVEY**

#### **2.1 FEASIBILITY STUDY**

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

#### **2.2 ECONOMICAL FEASIBILITY**

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

## **2.3 TECHNICAL FEASIBILITY**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

## **2.4 SOCIAL FEASIBILITY**

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

## CHAPTER 3

### SYSTEM ANALYSIS

#### 3.1 EXISTING SYSTEM:

- ❖ Many existing NLP techniques heavily rely on linguistic features, such as POS tags of the surrounding words, word capitalization, trigger words (e.g., Mr., Dr.), and gazetteers. These linguistic features, together with effective supervised learning algorithms (e.g., hidden markov model (HMM) and conditional random field (CRF)), achieve very good performance on formal text corpus. However, these techniques experience severe performance deterioration on tweets because of the noisy and short nature of the latter.
- ❖ In Existing System, to improve POS tagging on tweets, Ritter et al. train a POS tagger by using CRF model with conventional and tweet-specific features. Brown clustering is applied in their work to deal with the ill-formed words.

#### 3.2 DISADVANTAGES OF EXISTING SYSTEM:

- ❖ Given the limited length of a tweet (i.e., 140 characters) and no restrictions on its writing styles, tweets often contain grammatical errors, misspellings, and informal abbreviations.
- ❖ The error-prone and short nature of tweets often make the word-level language models for tweets less reliable.

### 3.3 PROPOSED SYSTEM:

In this paper, we focus on the task of tweet segmentation. The goal of this task is to split a tweet into a sequence of consecutive n-grams, each of which is called a segment. A segment can be a named entity (e.g., a movie title “finding nemo”), a semantically meaningful information unit (e.g., “officially released”), or any other types of phrases which appear “more than by chance”

- ❖ To achieve high quality tweet segmentation, we propose a generic tweet segmentation framework, named HybridSeg. HybridSeg learns from both global and local contexts, and has the ability of learning from pseudo feedback.
- ❖ Global context. Tweets are posted for information sharing and communication. The named entities and semantic phrases are well preserved in tweets.
- ❖ Local context. Tweets are highly time-sensitive so that many emerging phrases like “She Dancin” cannot be found in external knowledge bases.

### 3.4 ADVANTAGES OF PROPOSED SYSTEM:

- ✓ Our work is also related to entity linking (EL). EL is to identify the mention of a named entity and link it to an entry in a knowledge base like Wikipedia.

## **CHAPTER 4**

### **SYSTEM SPECIFICATIONS**

#### **4.1 Hardware Requirements:**

- System : Pentium IV 3.4 GHz.
- Hard Disk : 40 GB.
- Floppy Drive : 1.44 Mb.
- Monitor : 14' Colour Monitor.
- Mouse : Optical Mouse.
- Ram : 1 GB.

#### **4.2 Software Requirements:**

- Operating system : Windows Family.
- Coding Language : J2EE (JSP,Servlet,Java Bean)
- Data Base : MS Access.
- Web Server : Tomcat 6.0

### 4.3 PRELIMINARY INVESTIGATION

The first and foremost strategy for development of a project starts from the thought of designing a mail enabled platform for a small firm in which it is easy and convenient of sending and receiving messages, there is a search engine ,address book and also including some entertaining games. When it is approved by the organization and our project guide the first activity, ie. preliminary investigation begins. The activity has three parts:

- **Request Clarification**
- **Feasibility Study**
- **Request Approvals**

### 4.4 REQUEST CLARIFICATION

After the approval of the request to the organization and project guide, with an investigation being considered, the project request must be examined to determine precisely what the system requires.

Here our project is basically meant for users within the company whose systems can be interconnected by the Local Area Network(LAN). In today's busy schedule man need everything should be provided in a readymade manner. So taking into consideration of the vastly use of the net in day to day life, the corresponding development of the portal came into existence.

### 4.5 FEASIBILITY ANALYSIS

An important outcome of preliminary investigation is the determination that the system request is feasible. This is possible only if it is feasible within limited resource and time. The different feasibilities that have to be analyzed are

- **Operational Feasibility**
- **Economic Feasibility**
- **Technical Feasibility**



#### **4.5.1 Operational Feasibility**

Operational Feasibility deals with the study of prospects of the system to be developed. This system operationally eliminates all the tensions of the Admin and helps him in effectively tracking the project progress. This kind of automation will surely reduce the time and energy, which previously consumed in manual work. Based on the study, the system is proved to be operationally feasible.

#### **4.5.2 Economic Feasibility**

Economic Feasibility or Cost-benefit is an assessment of the economic justification for a computer based project. As hardware was installed from the beginning & for lots of purposes thus the cost on project of hardware is low. Since the system is a network based, any number of employees connected to the LAN within that organization can use this tool from at anytime.

#### **4.5.3 Technical Feasibility**

According to Roger S. Pressman, Technical Feasibility is the assessment of the technical resources of the organization. The organization needs IBM compatible machines with a graphical web browser connected to the Internet and Intranet. The system is developed for platform Independent environment. Java Server Pages, JavaScript, HTML, SQL server and Web Logic Server are used to develop the system. The technical feasibility has been carried out.

### **4.6 REQUEST APPROVAL**

Not all request projects are desirable or feasible. Some organization receives so many project requests from client users that only few of them are pursued. However, those projects that are both feasible and desirable should be put into schedule. After a project request is approved..

#### **4.6.1 SYSTEM REQUIREMENT:**

This Chapter describes about the requirements. It specifies the hardware and software requirements that are required in order to run the application properly. The Software Requirement Specification (SRS) is explained in detail, as the functional and non-functional requirement of this dissertation.

**SRS for**

|                           |                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functional</b>         | Admin login by using valid user name & password, admin can view the user details, view req and res users, Data sharing between one user to another user, user login by using authorized user name & password, Sending friend request to user, send anomaly messages, message Privacy Management, End user authentication by admin. |
| <b>Non- Functional</b>    | Admin never monitors the user activities                                                                                                                                                                                                                                                                                           |
| <b>External interface</b> | LAN , WAN                                                                                                                                                                                                                                                                                                                          |
| <b>Performance</b>        | Admin login, User login, search users, send friend request, send anomaly message, send Tweet Segmentation Topic messages, view users & mobile users, request, response, view user details.                                                                                                                                         |
| <b>Attributes</b>         | Topic Detection, Anomaly Detection, Social Networks, Twitter stream, tweet segmentation, named entity recognition, linguistic processing, Wikipedia, Admin, users.                                                                                                                                                                 |

Table: 3.1 Summaries of SRS

**4.6.2 Functional Requirements**

Functional Requirement defines a function of a software system and how the system must behave when presented with specific inputs or conditions. These may include calculations, data manipulation and processing and other specific functionality. In this system following are the functional requirements:-

- The Admin has to login by using valid user name and password.
- After login successful he can do some operations such as search history, view users, request & response, all topic messages and topics.
- The admin can view the search history details. If he clicks on search history button, it will show the list of searched user details with their tags such as user name, searched user, time and date.

- The admin can view the all the friend request and response. Here all the request and response will be stored with their tags such as Id, requested user photo, requested user name, user name request to, status and time & date.
- The admin can view the messages such as emerging topic messages and Tweet segmentation topic messages.
- The user can search the users based on users and the server will give response to the user like User name, user image, E mail id, phone number and date of birth.
- User can view the messages, send messages and Tweet messages to users. User can send messages based on topic to the particular user, after sending a message that topic rank will be increased.
- We can easily use android application. This application user has to install in a mobile. Before using this application user should register, after registration he should login by using authorized user name and password.
- After login successful he will do some operations such as view user Tweets, view Tweet segmentation topic messages, view all users, view request and response.
- The Attributes are Topic Detection, Social Networks, Twitter stream, tweet segmentation, named entity recognition, linguistic processing, Wikipedia, Admin, users.

#### **4.6.3 Non – Functional Requirements**

Non – Functional requirements, as the name suggests, are those requirements that are not directly concerned with the specific functions delivered by the system. They may relate to emergent system properties such as reliability response time and store occupancy. Alternatively, they may define constraints on the system such as the capability of the Input Output devices and the data representations used in system interfaces. Many non-functional requirements relate to the system as whole rather than to individual system features. This means they are often critical than the individual functional requirements. The following non-functional requirements are worthy of attention.

##### **The key non-functional requirements are:**

- **Security:** The system should allow a secured communication between server, Admin and users.

- Energy Efficiency: The Energy consumed by the Users to receive the File information from the server and admin.
- Reliability: The system should be reliable and must not degrade the performance of the existing system and should not lead to the hanging of the system.

## **4.7 SYSTEM DESIGN AND DEVELOPMENT**

### **4.7.1 INPUT DESIGN**

Input Design plays a vital role in the life cycle of software development, it requires very careful attention of developers. The input design is to feed data to the application as accurate as possible. So inputs are supposed to be designed effectively so that the errors occurring while feeding are minimized. According to Software Engineering Concepts, the input forms or screens are designed to provide to have a validation control over the input limit, range and other related validations.

This system has input screens in almost all the modules. Error messages are developed to alert the user whenever he commits some mistakes and guides him in the right way so that invalid entries are not made. Let us see deeply about this under module design.

Input design is the process of converting the user created input into a computer-based format. The goal of the input design is to make the data entry logical and free from errors. The error in the input are controlled by the input design. The application has been developed in user-friendly manner. The forms have been designed in such a way during the processing the cursor is placed in the position where must be entered. The user is also provided with in an option to select an appropriate input from various alternatives related to the field in certain cases.

Validations are required for each data entered. Whenever a user enters an erroneous data, error message is displayed and the user can move on to the subsequent pages after completing all the entries in the current page.

#### **4.7.2 OUTPUT DESIGN**

The Output from the computer is required to mainly create an efficient method of communication within the company primarily among the project leader and his team members, in other words, the administrator and the clients. The output of VPN is the system which allows the project leader to manage his clients in terms of creating new clients and assigning new projects to them, maintaining a record of the project validity and providing folder level access to each client on the user side depending on the projects allotted to him. After completion of a project, a new project may be assigned to the client. User authentication procedures are maintained at the initial stages itself. A new user may be created by the administrator himself or a user can himself register as a new user but the task of assigning projects and validating a new user rests with the administrator only.

The application starts running when it is executed for the first time. The server has to be started and then the internet explorer is used as the browser. The project will run on the local area network so the server machine will serve as the administrator while the other connected systems can act as the clients. The developed system is highly user friendly and can be easily understood by anyone using it even for the first time.

## CHAPTER 5

### Software Environment

#### 5.1 Java Technology

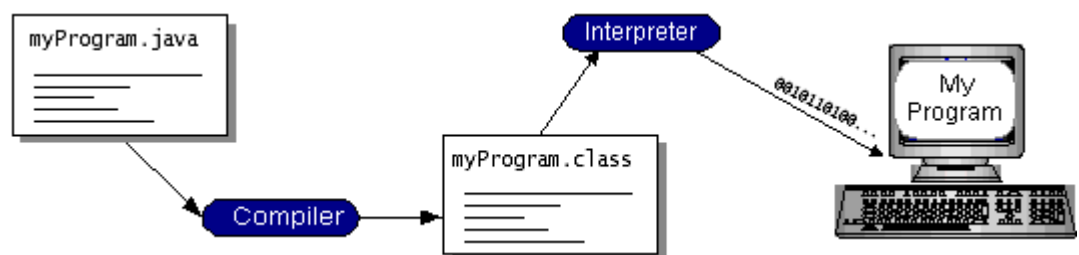
Java technology is both a programming language and a platform.

##### 5.1.1 The Java Programming Language

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

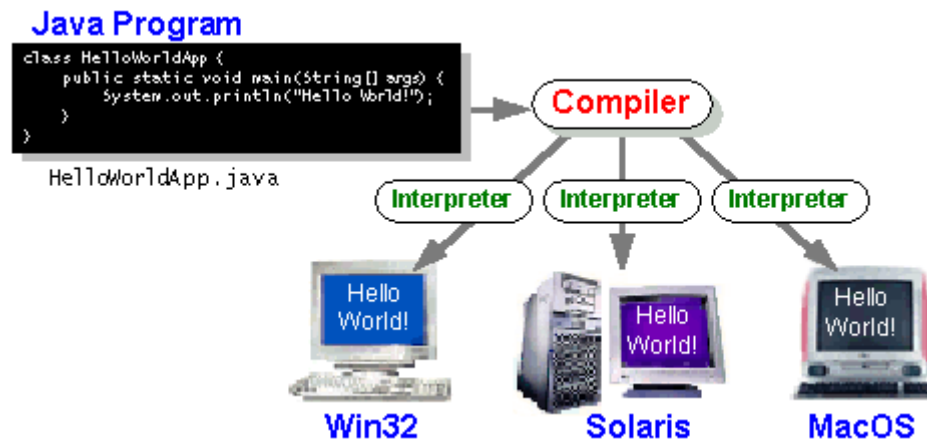
With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted.



5.1.1 Working Of Java

You can think of Java byte codes as the machine code instructions for the *Java Virtual Machine* (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java

byte codes help make “write once, run anywhere” possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.



### 5.1.1 The Java Programming

### 5.1.2 The Java Platform

A platform is the hardware or software environment in which a program runs. We’ve already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the operating system and hardware. The Java platform differs from most other platforms in that it’s a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

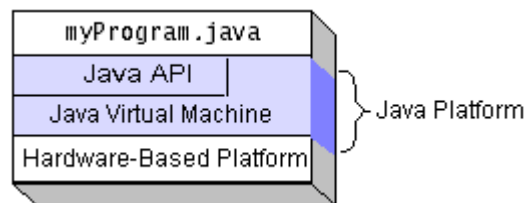
- The Java Virtual Machine (Java VM)
- The Java Application Programming Interface (s)

You’ve already been introduced to the Java VM. It’s the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and

interfaces; these libraries are known as *packages*. The next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.



### 5.1.2 Java platform

Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability.

### 5.1.3 What Can Java Technology Do?

The most common types of programs written in the Java programming language are . If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

An application is a standalone program that runs directly on the Java platform. A special kind of application known as a *server* serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a *servlet*. A servlet can almost be thought of as an applet that runs on the server side. Java Servlets are

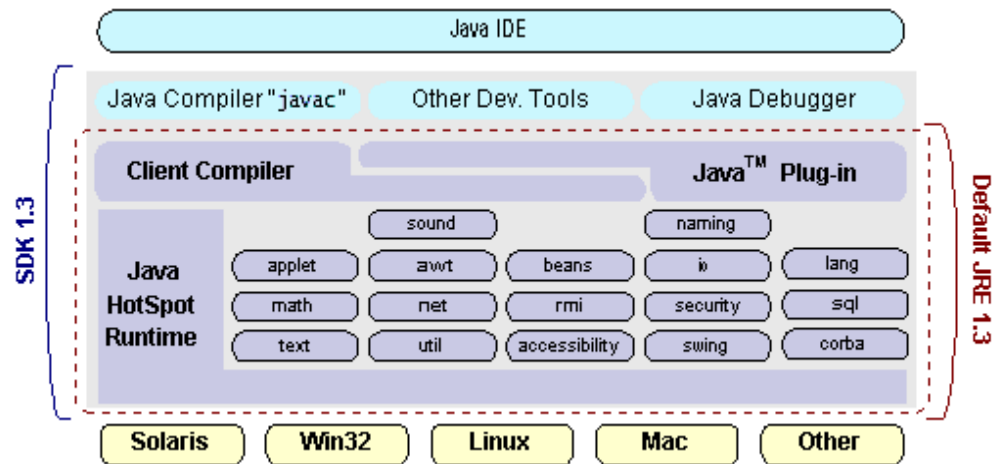


a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials:** Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets:** The set of conventions used by applets.
- **Networking:** URLs, TCP (Transmission Control Protocol), UDP (User Data gram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization:** Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.
- **Security:** Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components:** Known as JavaBeans<sup>TM</sup>, can plug into existing component architectures.
- **Object serialization:** Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBC<sup>TM</sup>):** Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.



### 5.1.3 Knowledge Discovery Process

### 5.1.4 How Will Java Technology Change My Life?

We can't promise you fame, fortune, or even a job if you learn the Java programming language. Still, it is likely to make your programs better and requires less effort than other languages. We believe that Java technology will help you do the following:

- **Get started quickly:** Although the Java programming language is a powerful object-oriented language, it's easy to learn, especially for programmers already familiar with C or C++.
- **Write less code:** Comparisons of program metrics (class counts, method counts, and so on) suggest that a program written in the Java programming language can be four times smaller than the same program in C++.
- **Write better code:** The Java programming language encourages good coding practices, and its garbage collection helps you avoid memory leaks. Its object orientation, its JavaBeans component architecture, and its wide-ranging, easily extendible API let you reuse other people's tested code and introduce fewer bugs.
- **Develop programs more quickly:** Your development time may be as much as twice as fast versus writing the same program in C++. Why? You write fewer lines of code and it is a simpler programming language than C++.

- **Avoid platform dependencies with 100% Pure Java:** You can keep your program portable by avoiding the use of libraries written in other languages. The 100% Pure Java™ Product Certification Program has a repository of historical process manuals, white papers, brochures, and similar materials online.
- **Write once, run anywhere:** Because 100% Pure Java programs are compiled into machine-independent byte codes, they run consistently on any Java platform.
- **Distribute software more easily:** You can upgrade applets easily from a central server. Applets take advantage of the feature of allowing new classes to be loaded “on the fly,” without recompiling the entire program.

## 5.2 ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a *de facto* standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is

a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data sources.

From a programming perspective, the beauty of ODBC is that the application can be written to use the same set of function calls to interface with any data source, regardless of the database vendor. The source code of the application doesn't change whether it talks to Oracle or SQL Server. We only mention these two as an example. There are ODBC drivers available for several dozen popular database systems. Even Excel spreadsheets and plain text files can be turned into data sources. The operating system uses the Registry information written by ODBC Administrator to determine which low-level ODBC drivers are needed to talk to the data source (such as the interface to Oracle or SQL Server). The loading of the ODBC drivers is transparent to the ODBC application program. In a client/server environment, the ODBC API even handles many of the network issues for the application programmer.

The advantages of this scheme are so numerous that you are probably thinking there must be some catch. The only disadvantage of ODBC is that it isn't as efficient as talking directly to the native database interface. ODBC has had many detractors make the charge that it is too slow. Microsoft has always claimed that the critical factor in performance is the quality of the driver software that is used. In our humble opinion, this is true. The availability of good ODBC drivers has improved a great deal recently. And anyway, the criticism about performance is somewhat analogous to those who said that compilers would never match the speed of pure assembly language. Maybe not, but the compiler (or ODBC) gives you the opportunity to write cleaner programs, which means you finish sooner. Meanwhile, computers get faster every year.

### 5.3 JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of "plug-in" database connectivity modules, or *drivers*. If a database vendor wishes to have JDBC

support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC's framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution.

JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

### **5.3.1 JDBC Goals**

Few software packages are designed without goals in mind. JDBC is one that, because of its many goals, drove the development of the API. These goals, in conjunction with early reviewer feedback, have finalized the JDBC class library into a solid framework for building database applications in Java.

The goals that were set for JDBC are important. They will give you some insight as to why certain classes and functionalities behave the way they do. The eight design goals for JDBC are as follows:

#### **1. SQL Level API**

The designers felt that their main goal was to define a SQL interface for Java. Although not the lowest database interface level possible, it is at a low enough level for higher-level tools and APIs to be created. Conversely, it is at a high enough level for application programmers to use it confidently. Attaining this goal allows for future tool vendors to “generate” JDBC code and to hide many of JDBC's complexities from the end user.

#### **2. SQL Conformance**

SQL syntax varies as you move from database vendor to database vendor. In an effort to support a wide variety of vendors, JDBC will allow any query statement to be passed through it to the underlying database driver. This allows

the connectivity module to handle non-standard functionality in a manner that is suitable for its users.

### **3. JDBC must be implemental on top of common database interfaces**

The JDBC SQL API must “sit” on top of other common SQL level APIs. This goal allows JDBC to use existing ODBC level drivers by the use of a software interface. This interface would translate JDBC calls to ODBC and vice versa.

### **4. Provide a Java interface that is consistent with the rest of the Java system**

Because of Java’s acceptance in the user community thus far, the designers feel that they should not stray from the current design of the core Java system.

### **5. Keep it simple**

This goal probably appears in all software design goal listings. JDBC is no exception. Sun felt that the design of JDBC should be very simple, allowing for only one method of completing a task per mechanism. Allowing duplicate functionality only serves to confuse the users of the API.

### **6. Use strong, static typing wherever possible**

Strong typing allows for more error checking to be done at compile time; also, less error appear at runtime.

### **7. Keep the common cases simple**

Because more often than not, the usual SQL calls used by the programmer are simple SELECT’s, INSERT’s, DELETE’s and UPDATE’s, these queries should be simple to perform with JDBC. However, more complex SQL statements should also be possible.

Finally we decided to proceed the implementation using Java Networking.

And for dynamically updating the cache table we go for MS Access database.

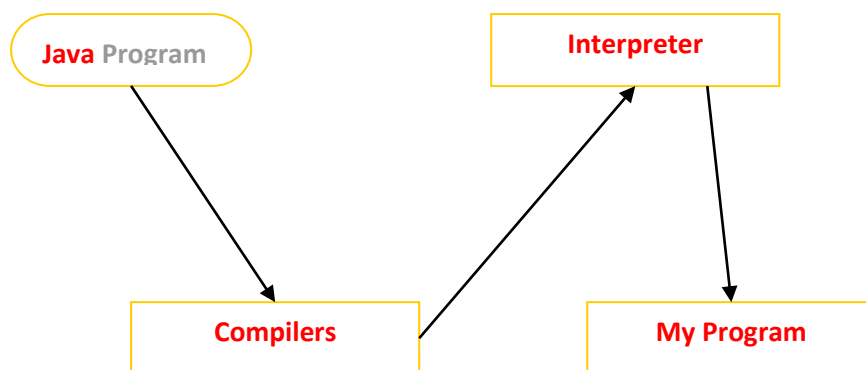
Java ha two things: a programming language and a platform.

Java is a high-level programming language that is all of the following

|                 |                      |
|-----------------|----------------------|
| Simple          | Architecture-neutral |
| Object-oriented | Portable             |
| Distributed     | High-performance     |
| Interpreted     | multithreaded        |
| Robust          | Dynamic              |
| Secure          |                      |

Java is also unusual in that each Java program is both compiled and interpreted. With a compile you translate a Java program into an intermediate language called Java byte codes the platform-independent code instruction is passed and run on the computer.

Compilation happens just once; interpretation occurs each time the program is executed. The figure illustrates how this works.



### 5.3.1 JDBC Compiler

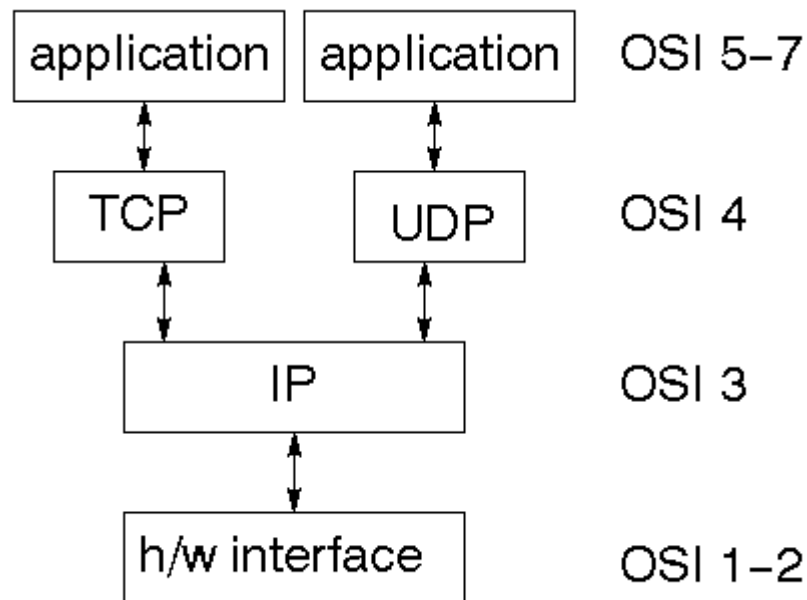
You can think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a Java development tool or a Web browser that can run Java applets, is an implementation of the Java VM.

Java byte codes help make “write once, run anywhere” possible. You can compile your Java program into byte codes on my platform that has a Java compiler.

## 5.4 Networking

### 5.4.1 TCP/IP stack

The TCP/IP stack is shorter than the OSI one:



5.4.1 TCP/IP Stack

### 5.4.2 IP datagram's

The IP layer provides a connectionless and unreliable delivery system. It considers each datagram independently of the others. Any association between datagram must be supplied by the higher layers. The IP layer supplies a checksum that includes its own header. The header includes the source and destination addresses, smaller ones for transmission and reassembling them at the other end.

## UDP

UDP is also connectionless and unreliable. What it adds to IP is a checksum for the contents of the datagram and port numbers. These are used to give a client/server model - see later.



## TCP

TCP supplies logic to give a reliable connection-oriented protocol above IP. It provides a virtual circuit that two processes can use to communicate.

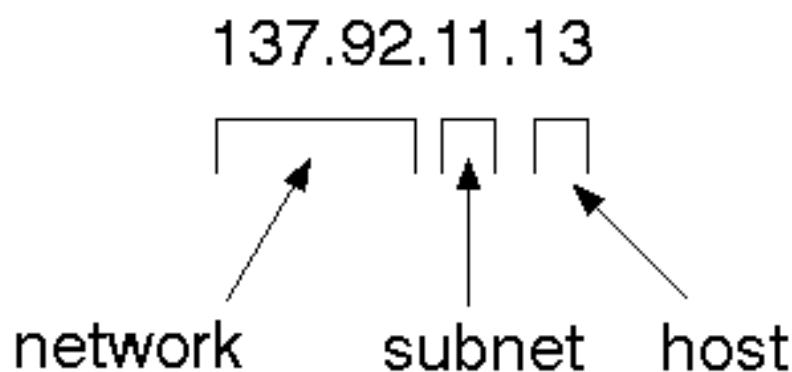
## Internet addresses

In order to use a service, you must be able to find it. The Internet uses an address scheme for machines so that they can be located. The address is a 32 bit integer which gives the IP address. This encodes a network ID and more addressing.

## Network address

Class A uses 8 bits for the network address with 24 bits left over for other addressing. Class B uses 16 bit network addressing. Class C uses 24 bit network addressing and class D uses all 32.

## Total address



The 32 bit address is usually written as 4 integers separated by dots.

## Port addresses

A service exists on a host, and is identified by its port. This is a 16 bit number. To send a message to a server, you send it to the port for that service of the host that it is running on. This is not location transparency! Certain of these ports are "well known".

## Sockets

A socket is a data structure maintained by the system to handle network connections. A socket is created using the call `socket`. It returns an integer that is like a file descriptor. In fact, under Windows, this handle can be used with `Read File` and `Write File` functions.

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int family, int type, int protocol);
```

Here "family" will be `AF_INET` for IP communications, `protocol` will be zero, and `type` will depend on whether TCP or UDP is used. Two processes wishing to communicate over a network create a socket each. These are similar to two ends of a pipe - but the actual pipe does not yet exist.

## 5.5 JFree Chart

JFreeChart is a free 100% Java chart library that makes it easy for developers to display professional quality charts in their applications. JFreeChart's extensive feature set includes:

- A flexible design that is easy to extend, and targets both server-side and client-side applications;

- Support for many output types, including Swing components, image files (including PNG and JPEG), and vector graphics file formats (including PDF, EPS and SVG);

- JFreeChart is "open source" or, more specifically, free software. It is distributed under the terms of the GNU Lesser General Public Licence (LGPL), which permits use in proprietary applications.

### 5.5.1 Map Visualizations

Charts showing values that relate to geographical areas. Some examples include: (a) population density in each state of the United States, (b)

income per capita for each country in Europe, (c) life expectancy in each country of the world. The tasks in this project include:

Testing, documenting, testing some more, documenting some more.

### **Time Series Chart Interactivity**

Implement a new (to JFreeChart) feature for interactive time series charts --- to display a separate control that shows a small version of ALL the time series data, with a sliding "view" rectangle that allows you to select the subset of the time series data to display in the main chart.

### **5.5.2 Dashboards**

There is currently a lot of interest in dashboard displays. Create a flexible dashboard mechanism that supports a subset of JFreeChart chart types (dials, pies, thermometers, bars, and lines/time series) that can be delivered easily via both Java Web Start and an applet.

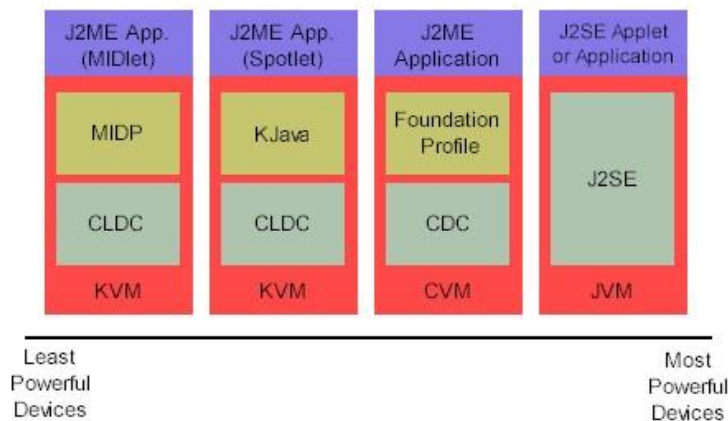
### **Property Editors**

The property editor mechanism in JFreeChart only handles a small subset of the properties that can be set for charts. Extend (or reimplement) this mechanism to provide greater end-user control over the appearance of the charts.

## **5.6 J2ME (Java 2 Micro edition):-**

Sun Microsystems defines J2ME as "a highly optimized Java run-time environment targeting a wide range of consumer products, including pagers, cellular phones, screen-phones, digital set-top boxes and car navigation systems." consumer products that incorporate or are based on small computing devices.

### 5.6.1 General J2ME architecture



### 5.6.1 General J2ME architecture

J2ME uses configurations and profiles to customize the Java Runtime Environment (JRE). As a complete JRE, J2ME is comprised of a configuration, which determines the JVM used, and a profile, which defines the application by adding domain-specific classes. The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. We'll discuss configurations in detail in the certain uses for devices. We'll cover profiles in depth in the The following graphic depicts the relationship between the different virtual machines, configurations, and profiles. It also draws a parallel with the J2SE API and its Java virtual machine. Both KVM and CVM can be thought of as a kind of Java virtual machine -- it's just that they are shrunken versions of the J2SE JVM and are specific to J2ME.

### 5.6.2 Developing J2ME applications

**Introduction** In this section, we will go over some considerations you need to keep in mind when developing applications for smaller devices. We'll take a look at the way the compiler is invoked

#### Design considerations for small devices

Developing applications for small devices requires you to keep certain strategies in mind during the design phase. It is best to strategically design an application for a

consider all of the "gotchas" before developing the application can be a painful process. Here are some design strategies to consider:

- \* Keep it simple. Remove unnecessary features, possibly making those features a separate, secondary application.

### 5.6.3 Configurations overview

The configuration defines the basic run-time environment as a set of core classes and a specific JVM that run on specific types of devices. Currently, two configurations exist for J2ME, though others may be defined in the future:

## 5.7 J2ME profiles

### 5.7.1 What is a J2ME profile?

As we mentioned earlier in this tutorial, a profile defines the type of device supported. The Mobile Information Device Profile (MIDP), for example, defines classes for cellular phones.

#### Profile 1: KJava

KJava is Sun's proprietary profile and contains the KJava API. The KJava profile is built on top of the CLDC configuration. The KJava virtual machine, KVM, accepts the same byte codes and class file format as the classic J2SE virtual machine. KJava contains a Sun-specific API that runs

#### Profile 2: MIDP

MIDP is geared toward mobile devices such as cellular phones and pagers. The MIDP, like KJava, is built upon CLDC and provides a standard run-time environment that allows new applications and services to be deployed dynamically on end user devices. MIDP is a common.

## CHAPTER 6

### DESIGN ANALYSIS

#### 6.1 UML Diagrams:

UML is a method for describing the system architecture in detail using the blueprint. UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. UML is a very important part of developing objects oriented software and the software development process. UML uses mostly graphical notations to express the design of software projects.

##### **Definition:**

UML is a general-purpose visual modeling language that is used to specify, visualize, construct, and document the artifacts of the software system.

##### **UML is a language:**

It will provide vocabulary and rules for communications and function on conceptual and physical representation. So it is modeling language.

##### **UML Specifying:**

Specifying means building models that are precise, unambiguous and complete. In particular, the UML address the specification a software intensive system.

##### **UML Visualization:**

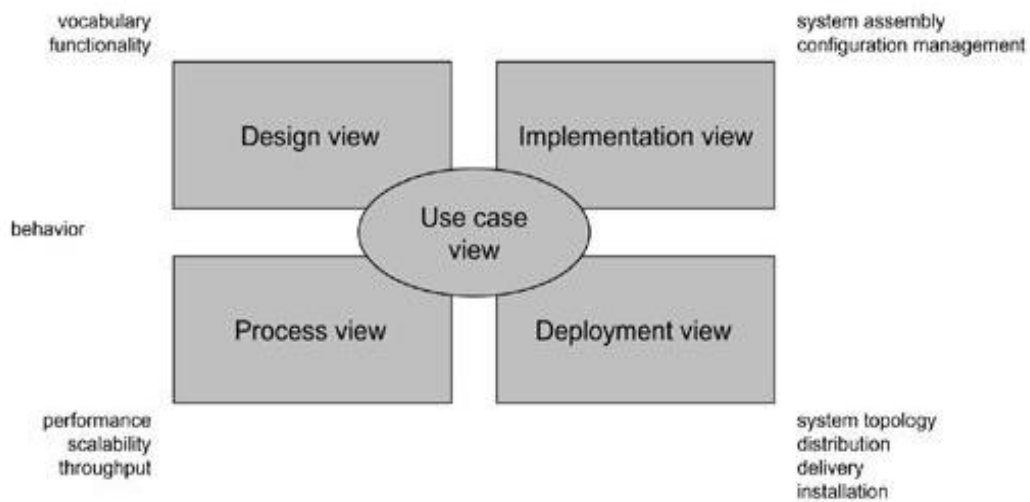
The UML includes both graphical and textual representation. It makes easy to visualize the system and for better understanding.

##### **UML Constructing:**

UML models can be directly connected to a variety of programming languages and it is sufficiently expressive and free from any ambiguity to permit the direct execution of models.

##### **UML Documenting:**

UML provides variety of documents in addition raw executable codes.



**Figure 6.1 Modeling a System Architecture using views of UML**

The **use case view** of a system encompasses the use cases that describe the behavior of the system as seen by its end users, analysts, and testers.

The **design view** of a system encompasses the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution.

The **process view** of a system encompasses the threads and processes that form the system's concurrency and synchronization mechanisms.

The **implementation view** of a system encompasses the components and files that are used to assemble and release the physical system.

The **deployment view** of a system encompasses the nodes that form the system's hardware topology on which the system executes.

#### Uses of UML:

The UML is intended primarily for software intensive systems. It has been used effectively for such domain as

- Enterprise Information System
- Banking and Financial Services
- Telecommunications
- Transportation
- Defense/Aerospace
- Retails
- Medical Electronics
- Scientific Fields
- Distributed Web

## 6.2 Building blocks of UML:

The vocabulary of the UML encompasses 3 kinds of building blocks

- Things
- Relationships
- Diagrams

### Things:

Things are the data abstractions that are first class citizens in a model. Things are of 4 types

Structural Things, Behavioral Things, Grouping Things, A notational Thing

### Relationships:

Relationships tie the things together. Relationships in the UML are

Dependency, Association, Generalization, Specialization

### UML Diagrams:

A diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).

There are two types of diagrams, they are: Structural Diagrams and Behavioral Diagrams

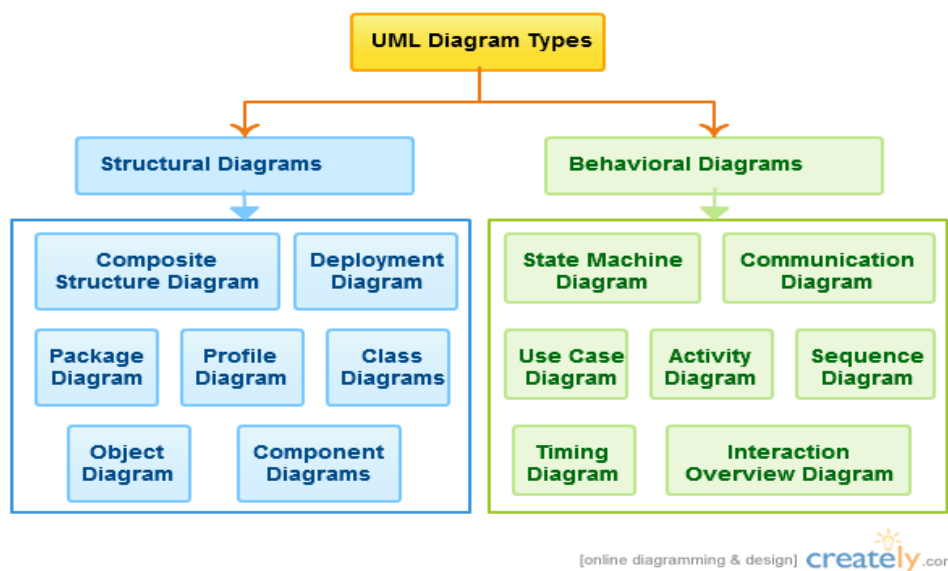


Figure 6.2 UML Diagrams Types



**Structural Diagrams:**

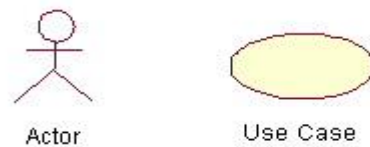
The UML's four structural diagrams exist to visualize, specify, construct and document the static aspects of a system. Structural diagrams consist of Class Diagram, Object Diagram, Component Diagram, and Deployment Diagram.

**Behavioral Diagrams:**

The UML's five behavioral diagrams are used to visualize, specify, construct, and document the dynamic aspects of a system.

**6.2.1 USE-CASE DIAGRAM:**

A use case is a set of scenarios that describing an interaction between a user and a system. A use case diagram displays the relationship among actors and use

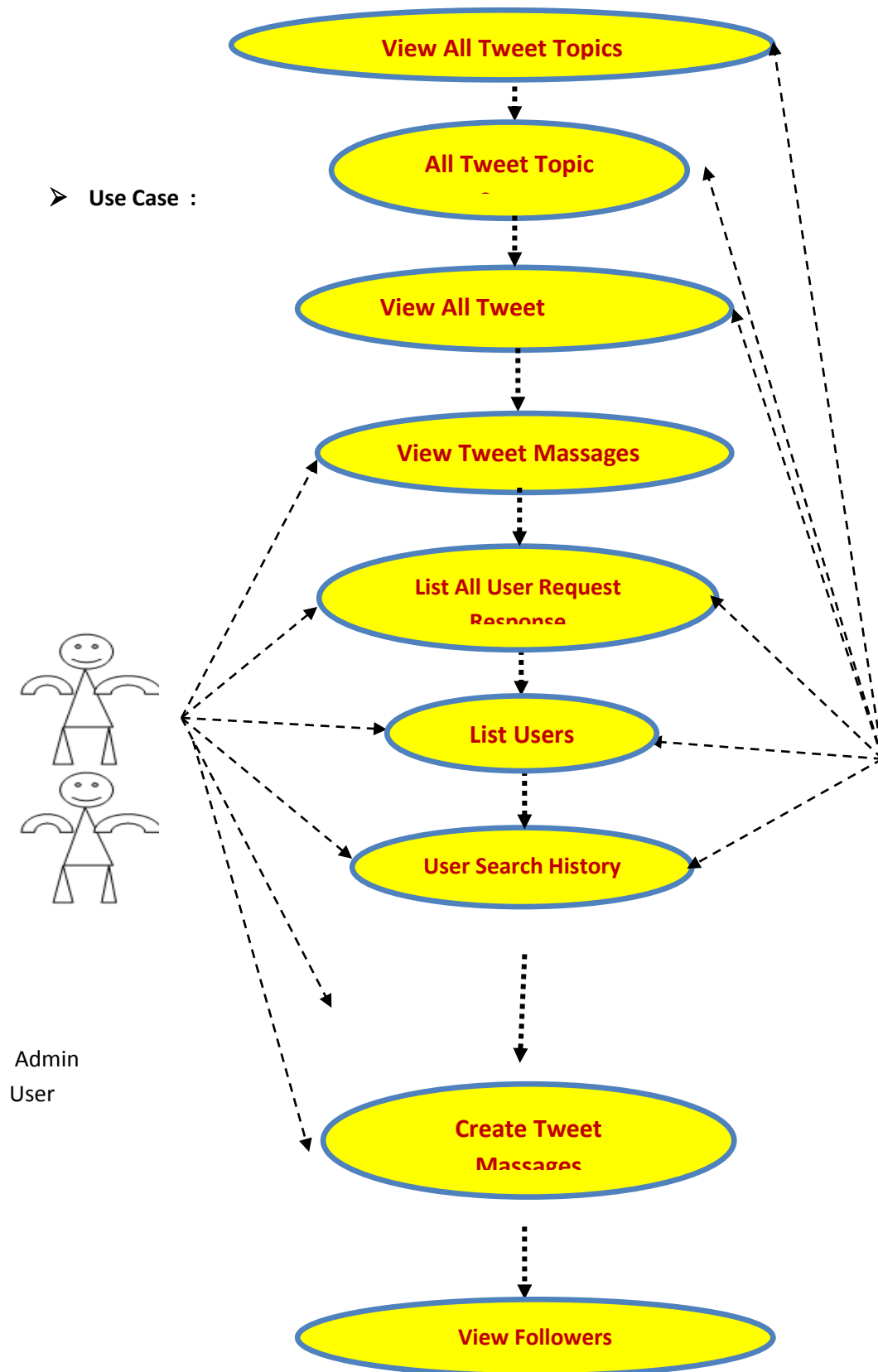


cases.

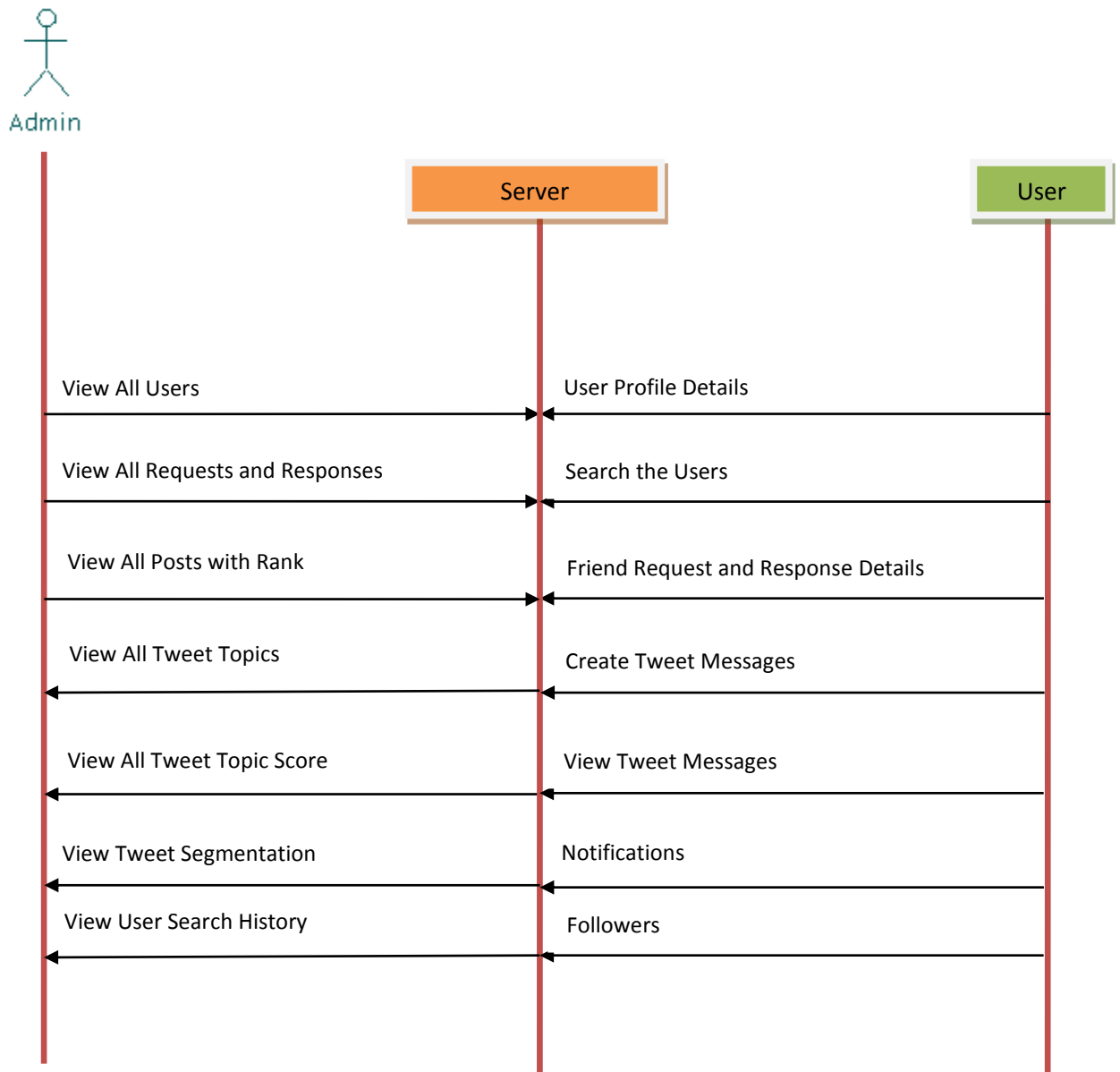
Contents:

- Use cases
- Actors
- Dependency, Generalization, and association relationships

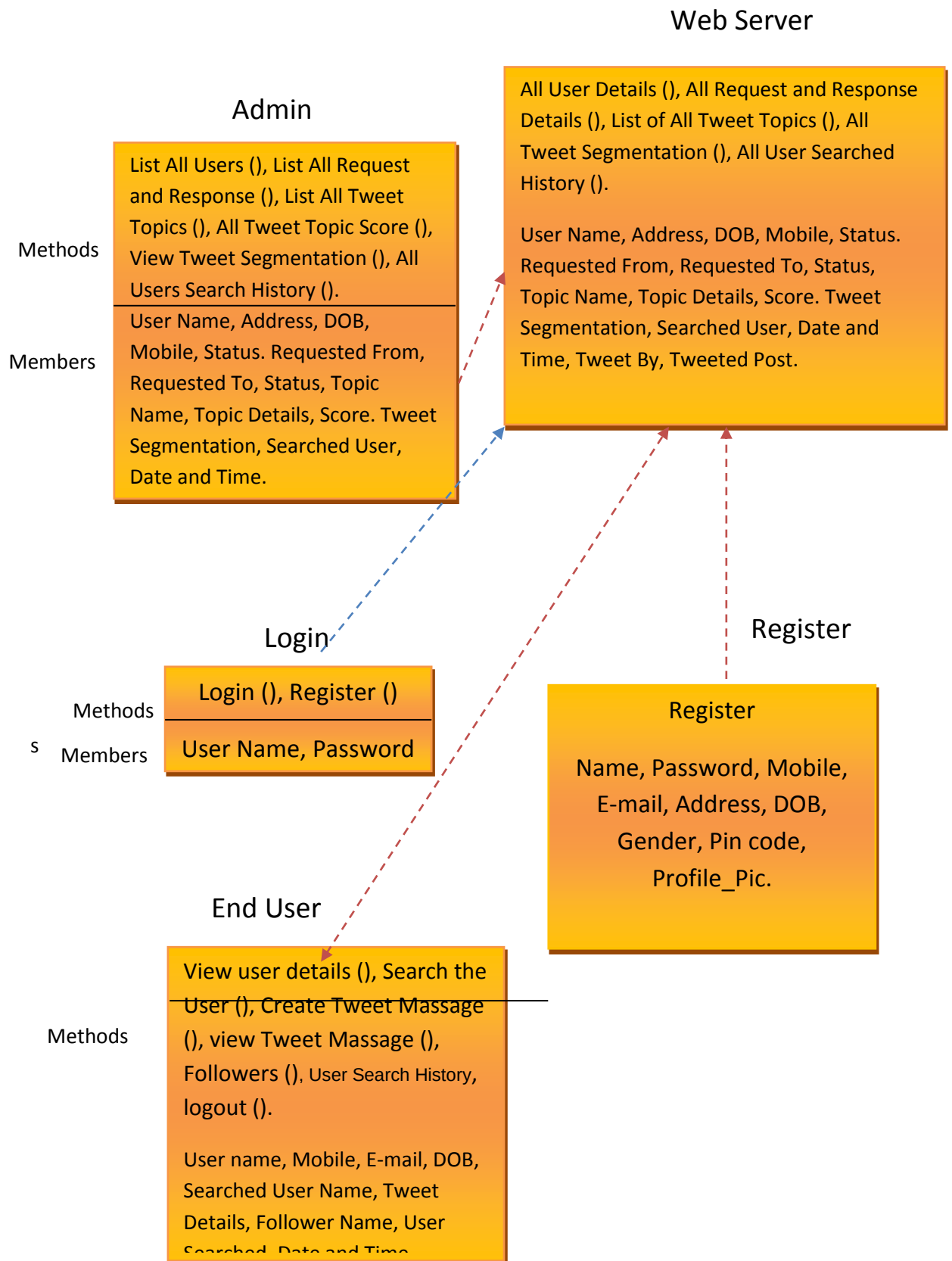
## USE CASE DIAGRAM

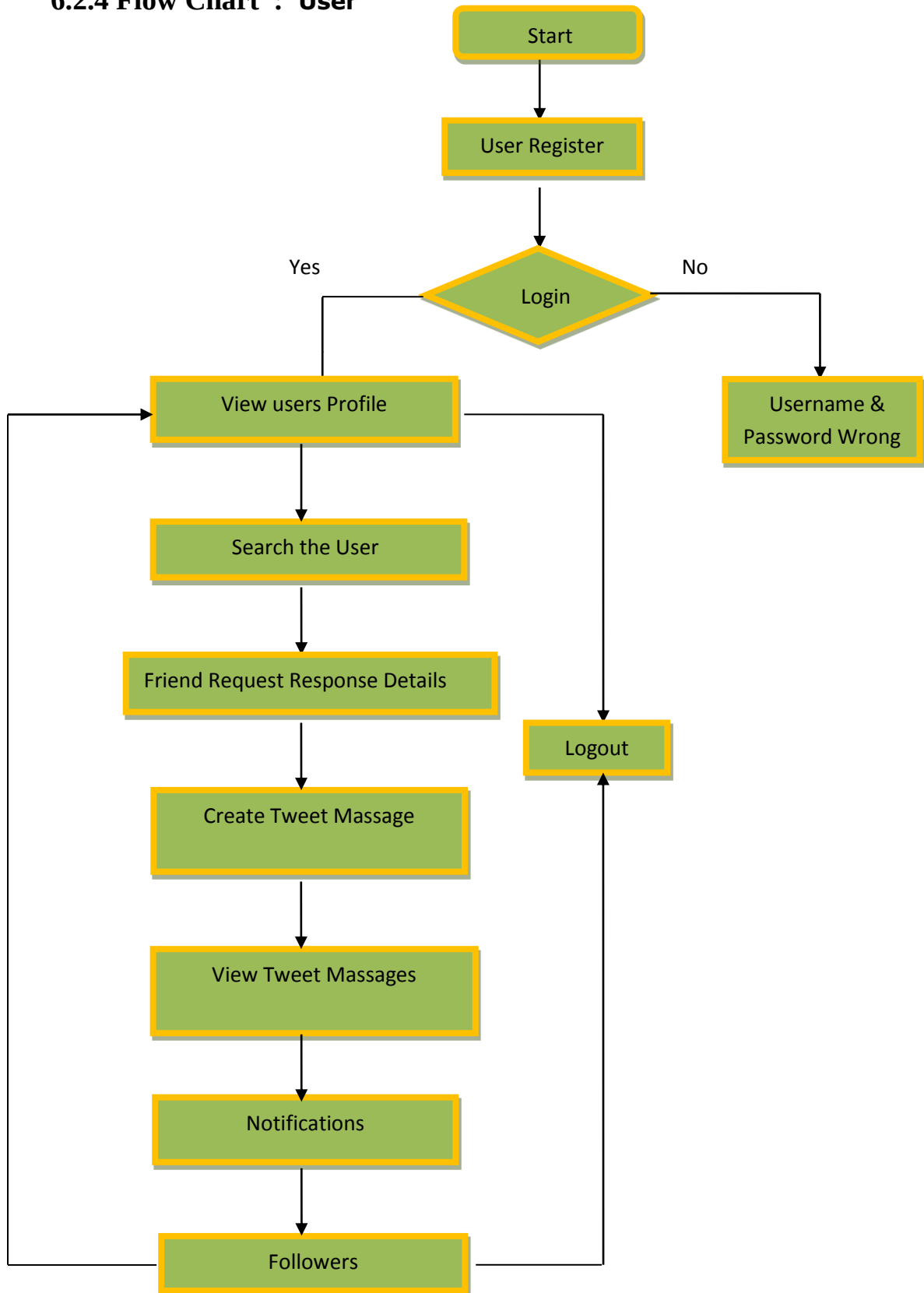


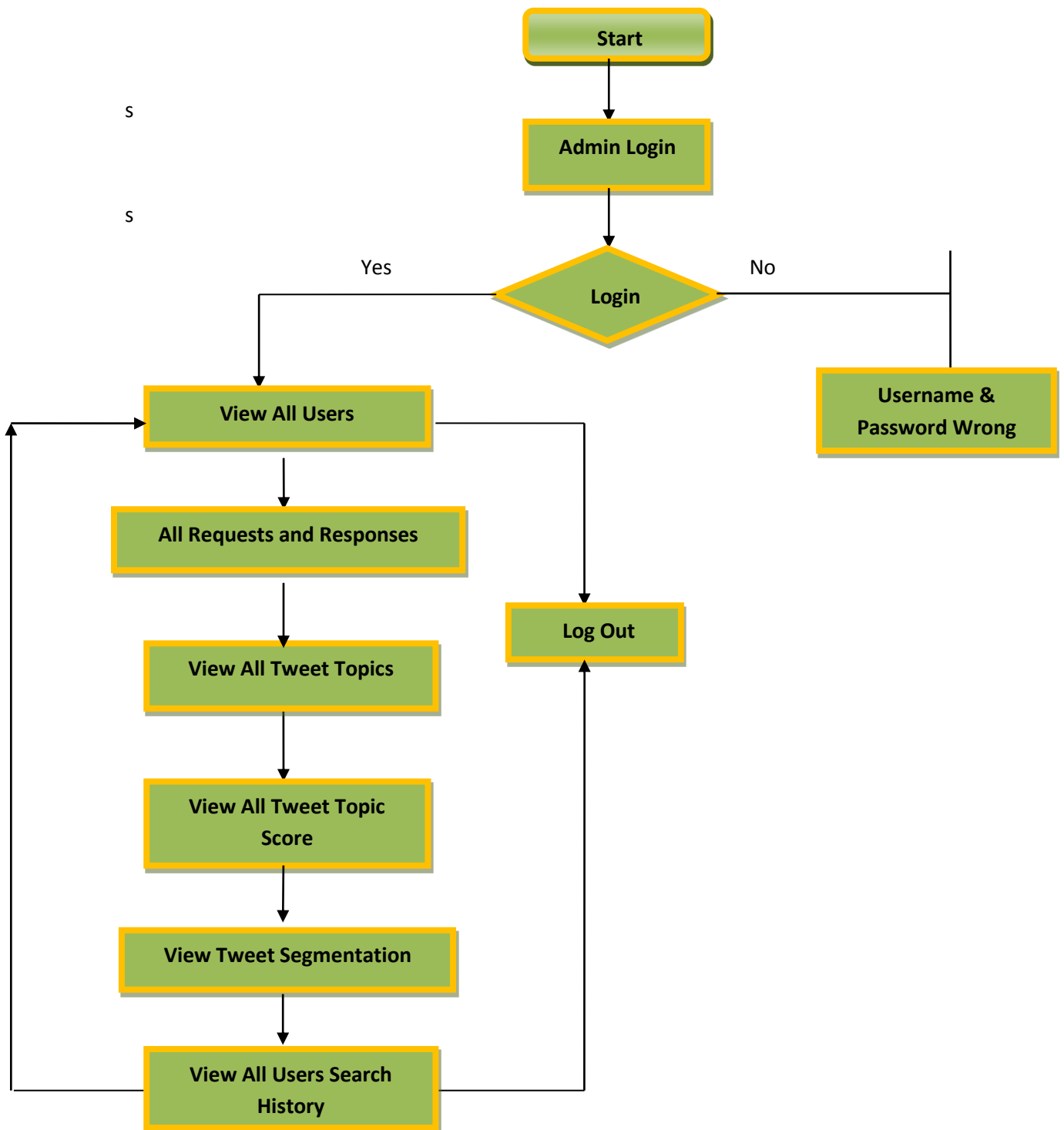
### 6.2.2 Sequence Diagram:



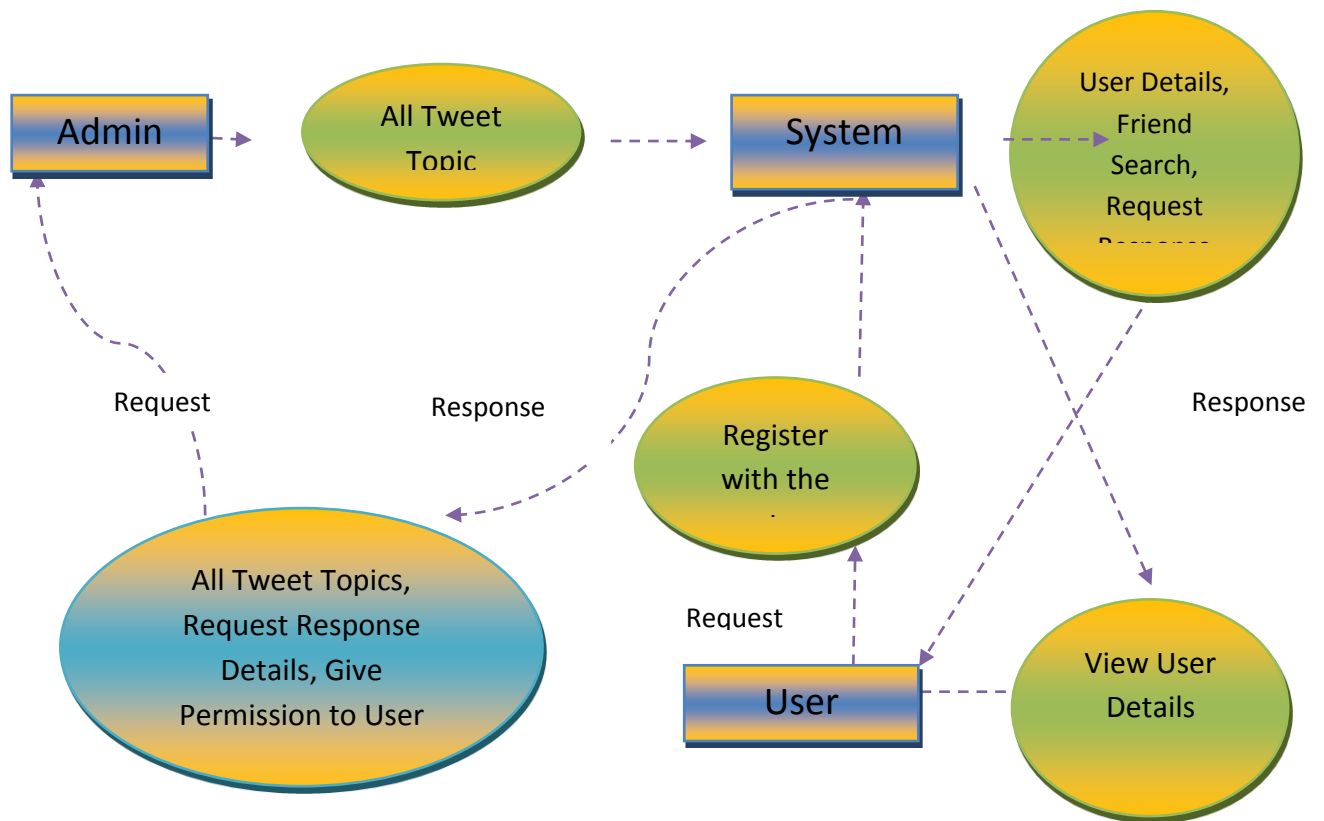
### 6.2.3 Class Diagram



**6.2.4 Flow Chart : User**

**Flow Chart : Admin**

### 6.2.5 Data Flow Diagram



## CHAPTER 7

### IMPLEMENTATION RESULTS

#### 7.1 Coding

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Home Page::Tweet Segmentation</title>

<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />

<link rel="stylesheet" type="text/css" href="css/style.css" />

<style type="text/css">

<!--

.style2 {

    font-size: 36px;

    color: #0000FF;

}

.style4 {

    color: #FF0000;

    font-size: 16px;

    font-weight: bold;

}

-->

</style>

</head>

<body>

<div id="wh_bg">

    <div id="bg_bg">

```



```

<div id="top">

  <div class="top1"></div>

  <div class="top2"> <a href="index.html"></a> </div>

  <div id="menu">

    <div class="men_tp"> <a href="index.html">HOME</a>  <a
href="Admin.html">ADMIN</a>  <a href="User.html">USER</a>  <a
href="Register.jsp">REGISTRATION</a>  <a href="Aboutus.html">ABOUT PROJECT </a>
 </div>

  </div>

</div>

<div id="content">

  <div id="part1">

    <div id="img_para"> <span class="img_tp"></span>    </div>

    <div id="con">

      <div class="con1">

        <div align="justify"><br />

        <br />

```

In this paper, the system proposes a novel framework for tweet segmentation in a batch mode, called HybridSeg. By splitting tweets into meaningful segments, the semantic or context information is well preserved and easily extracted by the downstream applications. HybridSeg finds the optimal segmentation of a tweet by maximizing the sum of the stickiness scores of its candidate segments. The stickiness score considers the probability of a segment being a phrase in English (i.e., global context) and the probability of a segment being a phrase within the batch of tweets (i.e., local context). For the latter, we propose and evaluate two models to derive local context by considering the linguistic features and term-

dependency in a batch of tweets respectively. HybridSeg is also designed to iteratively learn from confident segments as pseudo feedback. Experiments on two tweet data sets show that tweet segmentation quality is significantly improved by learning both global and local contexts compared with using global context alone. Through analysis and comparison, we show that local linguistic features are more reliable for learning local context compared with term-dependency. As an application, we show that high accuracy is achieved in named entity recognition by applying segment-based part-of-speech (POS) tagging.

</div>

<div class="con2">

<p><span class="flt" style="width:270px;"><span class="date\_txt style2">Concepts</span></span></p>

<p align="justify"><span class="flt" style="width:270px;"><span class="date\_txt"><br />

<span class="style4">1.Twitter stream,<br />

<br />

2.tweet segmentation,<br />

<br />

3.named entity recognition,<br />

<br />

4.linguistic processing, <br />

<br />

5.Wikipedia</span><br />

</span> </span> <br />

</p>

</div>

</div>

</div>

<div id="part2">

<div id="cen">

```
<div id="abt">
```

```

    <div align="justify"> <span class="abt_txt"> The system aims to propose a novel framework for tweet segmentation in a
batch mode, called HybridSeg. By splitting tweets into meaningful segments, the semantic or
context information is well preserved and easily extracted by the downstream applications.
HybridSeg finds the optimal segmentation of a tweet by maximizing the sum of the stickiness
scores of its candidate segments. The stickiness score considers the probability of a segment
being a phrase in English (i.e., global context) and the probability of a segment being a
phrase within the batch of tweets (i.e., local context). For the latter, we propose and
evaluate two models to derive local context by considering the linguistic features and term-
dependency in a batch of tweets, respectively.<br />

```

```
    <br />
```

```
    <br />
```

```
  </span> </div>
```

```
</div>
```

```
</div>
```

```
<div id="cen1">
```

```

    <div id="abt_prt"> 

```

```

    <div id="link"> <a href="http://all-free-download.com/free-website-templates/"></a>
<a href="http://all-free-download.com/free-website-templates/"></a> <a href="http://all-
free-download.com/free-website-templates/"></a> <a href="http://all-free-
download.com/free-website-templates/"></a> <a href="http://all-free-download.com/free-
website-templates/"></a> </div>

```

```

    <a href="http://all-free-download.com/free-website-templates/"
style="color:#FAFF6A;clear:left;float:left;width:250px; margin:10px 0px 0px 15px;"></a> <br
/>

```

```
    <br />
```

```
  </div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```

<div id="footer">

    <div id="foot"> <a href="http://all-free-download.com/free-website-
templates/">Home</a> <span class="flt">|</span> <a href="http://all-free-
download.com/free-website-templates/">About Us</a> <span class="flt">|</span> <a
href="http://all-free-download.com/free-website-templates/">Services</a> <span
class="flt">|</span> <a href="http://all-free-download.com/free-website-templates/">Our
Media</a> <span class="flt">|</span> <a href="http://all-free-download.com/free-website-
templates/">Our Clients</a> <span class="flt">|</span> <a href="http://all-free-
download.com/free-website-templates/">Solutions</a> <span class="flt">|</span> <a
href="http://all-free-download.com/free-website-templates/">Contact Us</a> </div>

    </div>

</div>

</div>

<div align=center></div>

</body>

</html>

```

## 7.2 Screen shots:



Technical Details

View Search History Details !!!

| User Name | Search User | Date & Time         |
|-----------|-------------|---------------------|
| test      | test        | 03/10/2015 18:18:32 |
| test      | test        | 03/10/2015 18:18:32 |
| test      | test        | 03/10/2015 18:19:34 |
| test      | test        | 03/10/2015 18:19:35 |
| test      | test        | 03/10/2015 18:20:15 |
| test      | test        | 03/10/2015 18:20:52 |
| test      | test        | 03/10/2015 18:27:08 |
| Harish    | test        | 03/10/2015 18:38:34 |
| Harish    | test        | 03/10/2015 18:38:34 |
| Harish    | test        | 03/10/2015 18:38:34 |
| Mohana    | test        | 03/10/2015 18:38:34 |
| test      | Harish      | 03/10/2015 18:38:34 |
| test      | Mohana      | 03/10/2015 18:38:34 |
| test      | Mohana      | 03/10/2015 18:38:34 |
| test      | Harish      | 03/10/2015 18:38:34 |

[Back](#)

[ABOUT PROJECT](#)

The system aims to propose a novel framework for tweet segmentation in a batch mode, called Hybridizing. By splitting tweets into meaningful segments, the semantic or content information is well preserved and easily extracted by the downstream applications. Hybridizing finds the optimal segmentation of a tweet by maximizing the sum of the stickiness scores of its candidate segments. The stickiness score is defined as the ratio of the number of words in the segment to the total number of words in the tweet.

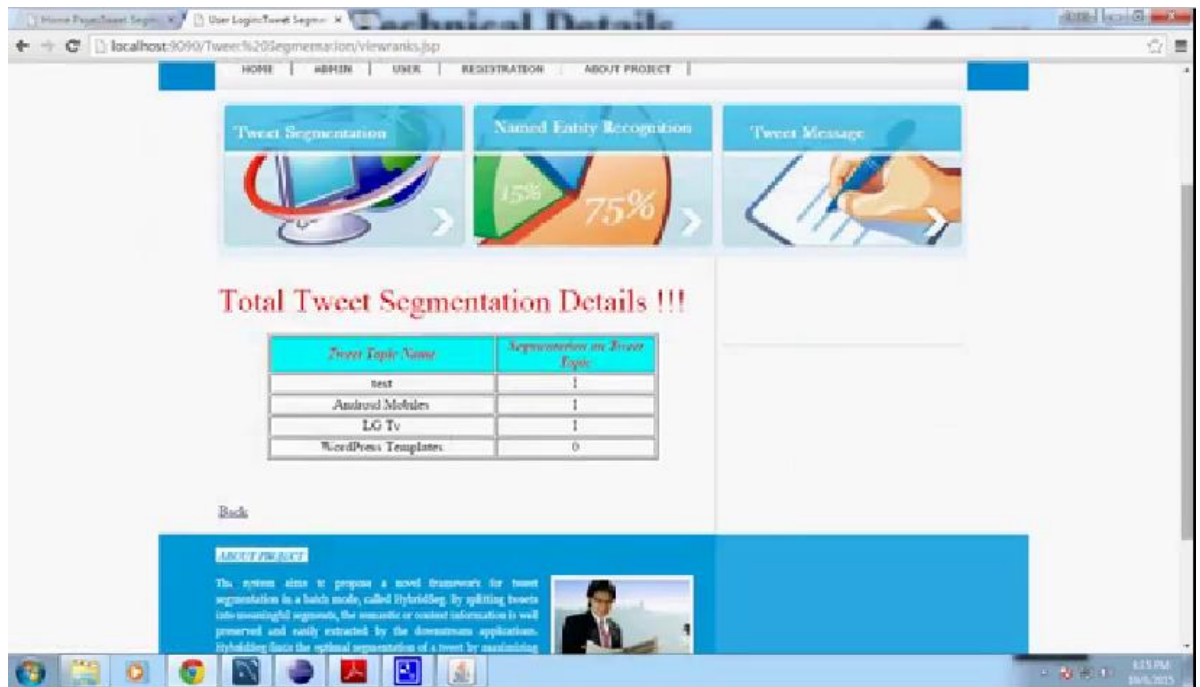
Technical Details

List of All Users's Link !!! [Back](#)

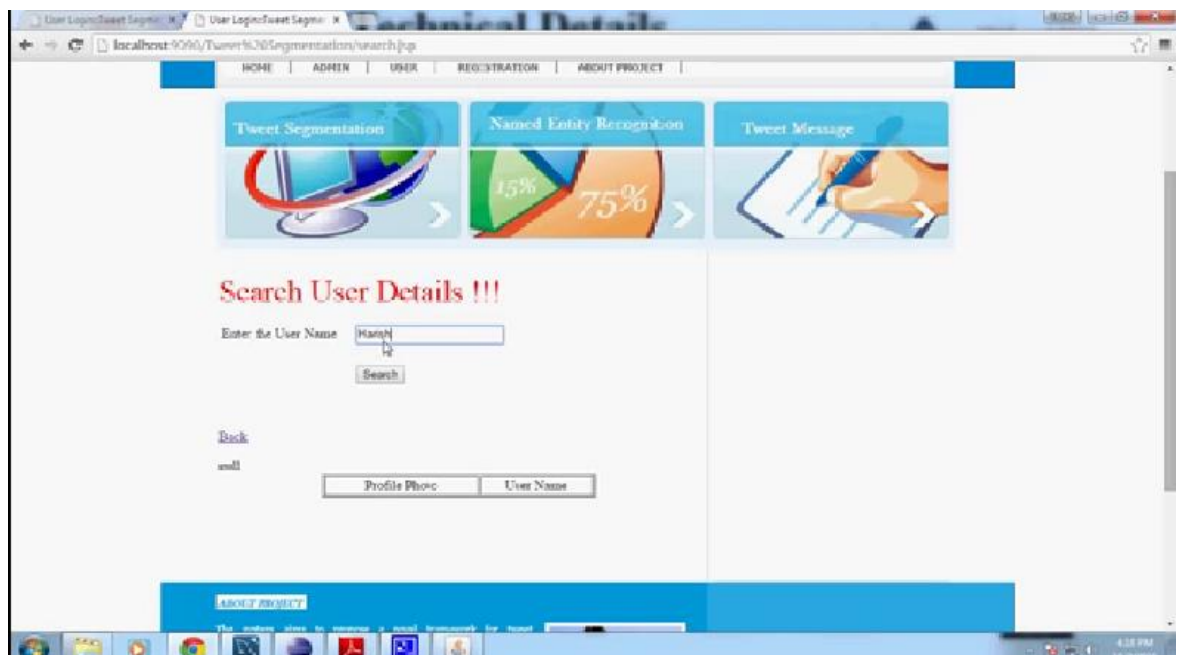
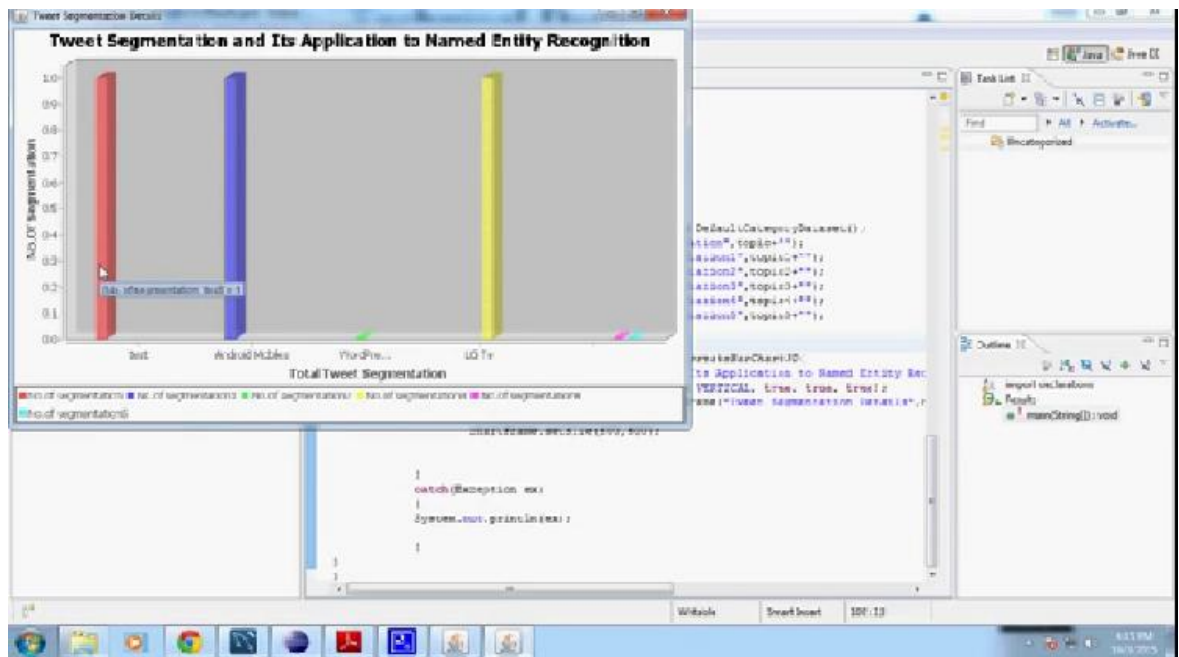
| Id | Requested User                                                                      | Requested User Name | User Request To                                                                     | User Name Request To | Status   | Date & Time         |
|----|-------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------|----------------------|----------|---------------------|
| 1  |  | test                |  | test                 | Accepted | 03/10/2015 18:19:25 |
| 2  |  | Harish              |  | test                 | Accepted | 03/10/2015 18:19:04 |
| 3  |  | test                |  | Mohan                | Accepted | 03/10/2015 18:38:52 |

[ABOUT PROJECT](#)

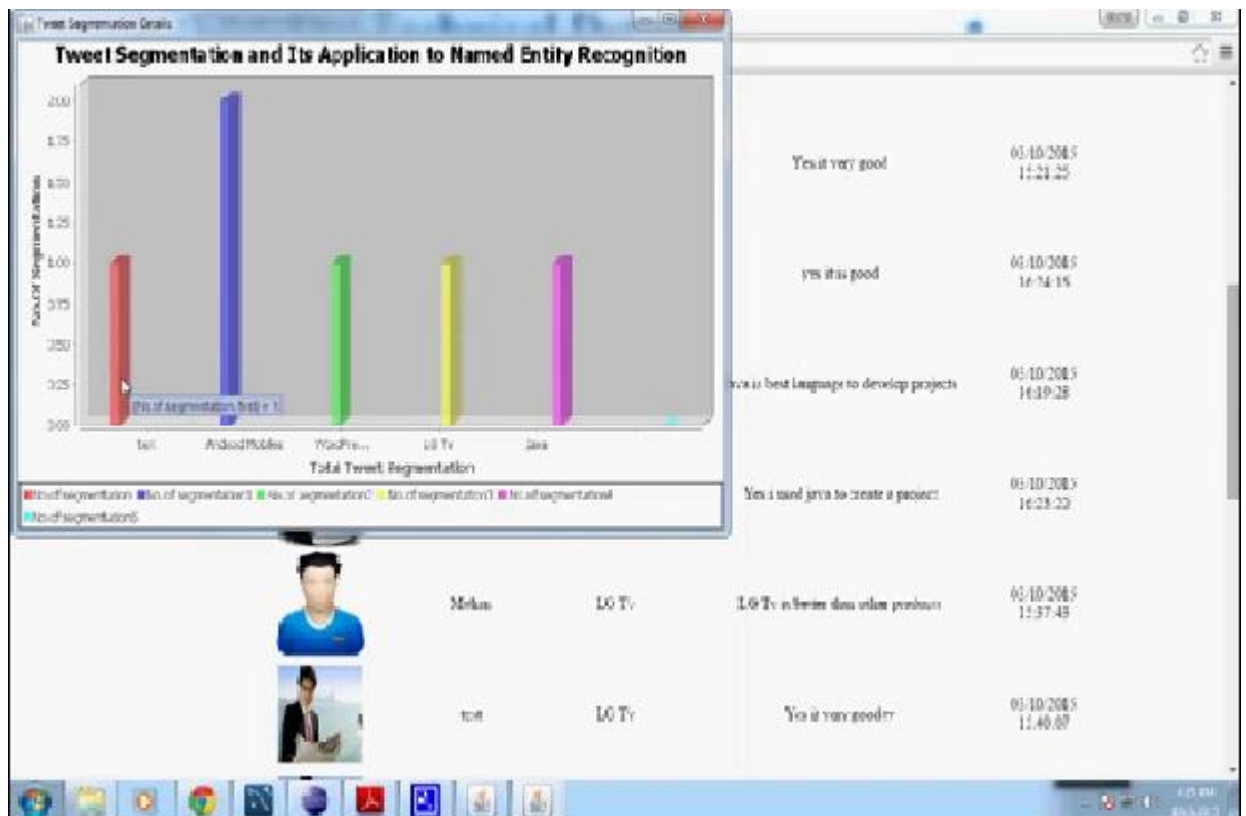
The system aims to propose a novel framework for tweet segmentation in a batch mode, called Hybridizing. By splitting tweets into meaningful segments, the semantic or content information is well preserved and easily extracted by the downstream applications. Hybridizing finds the optimal segmentation of a tweet by maximizing the sum of the stickiness scores of its candidate segments. The stickiness score is defined as the ratio of the number of words in the segment to the total number of words in the tweet.



## TWEET SEGMENTATION AND ITS NAMED ENTITY RECOGNITION







## 7.3 Implementations Results

- **Admin**

In this module, the Admin has to login by using valid user name and password. After login successful he can do some operations such as search history, view users, request & response, all topic messages and topics.

### Search History

This is controlled by admin; the admin can view the search history details. If he clicks on search history button, it will show the list of searched user details with their tags such as user name, searched user, time and date.

### Request & Response

In this module, the admin can view the all the friend request and response. Here all the request and response will be stored with their tags such as Id, requested user photo, requested user name, user name request to, status and time & date. If the user accepts the request then status is accepted or else the status is waiting.

### **Tweet segmentation Topic Messages**

In this module, the admin can view the messages such as emerging topic messages and Anomaly emerging topic messages. Tweet segmentation topic messages means we can send a message to particular user

### **User**

In this module, there are n numbers of users are present. User should register before doing some operations. And register user details are stored in user module. After registration successful he has to login by using authorized user name and password. Login successful he will do some operations like view or search users, send friend request, view messages, send messages, Tweet segmentation messages and followers.

### **Search Users**

The user can search the users based on users and the server will give response to the user like User name, user image, E mail id, phone number and date of birth. If you want send friend request to particular receiver then click on follow, then request will send to the user.

### **Messages**

User can view the messages, send messages and send anomaly messages to users. User can send messages based on topic to the particular user, after sending a message that topic rank will be increased. Then again another user will also re-tweet the particular topic then that topic rank will increases. The anomaly message means user wants send a message to all users.

### **Followers**

In this module, we can view the followers' details with their tags such as user name, user image, date of birth, E mail ID, phone number and ranks.

## **CHAPTER 8**

### **SYSTEM TESTING**

#### **8.1 Introduction of system testing**

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

#### **8.2 TYPES OF TESTS**

##### **Unit testing**

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

##### **Integration testing**

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

**Functional test**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

**System Test**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

**White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

**Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

**8.3 Unit Testing:**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

**Test strategy and approach**

Field testing will be performed manually and functional tests will be written in detail.

**Test objectives**

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

**Features to be tested**

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

### **Integration Testing**

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

**Test Results:** All the test cases mentioned above passed successfully. No defects encountered.

### **Acceptance Testing**

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

**Test Results:**

All the test cases mentioned above passed successfully. No defects encountered.

## **8.4 TESTING METHODOLOGIES**

The following are the Testing Methodologies:

- **Unit Testing.**
- **Integration Testing.**
- **User Acceptance Testing.**
- **Output Testing.**
- **Validation Testing.**

## **Unit Testing**

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to

ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing.

During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

## **Integration Testing**

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

### **The following are the types of Integration Testing:**

#### **1. Top Down Integration**

This method is an incremental approach to the construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main program module. The module subordinates to the main program module are incorporated into the structure in either a depth first or breadth first manner.

In this method, the software is tested from main module and individual stubs are replaced when the test proceeds downwards.

#### **2. Bottom-up Integration**

This method begins the construction and testing with the modules at the lowest level in the program structure. Since the modules are integrated from the bottom up,

processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. The bottom up integration strategy may be implemented with the following steps:

- The low-level modules are combined into clusters into clusters that perform a specific Software sub-function.
- A driver (i.e.) the control program for testing is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure

The bottom up approaches tests each module individually and then each module is module is integrated with a main module and tested for functionality.

### **User Acceptance Testing**

User Acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required. The system developed provides a friendly user interface that can easily be understood even by a person who is new to the system.

### **Output Testing**

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specified format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration. Hence the output format is considered in 2 ways – one is on screen and another in printed format.

### **Validation Checking**

Validation checks are performed on the following fields.



**Text Field:**

The text field can contain only the number of characters lesser than or equal to its size. The text fields are alphanumeric in some tables and alphabetic in other tables. Incorrect entry always flashes and error message.

**Numeric Field:**

The numeric field can contain only numbers from 0 to 9. An entry of any character flashes an error messages. The individual modules are checked for accuracy and what it has to perform. Each module is subjected to test run along with sample data. The individually tested modules are integrated into a single system. Testing involves executing the real data information is used in the program the existence of any program defect is inferred from the output. The testing should be planned so that all the requirements are individually tested.

**Preparation of Test Data**

Taking various kinds of test data does the above testing. Preparation of test data plays a vital role in the system testing. After preparing the test data the system under study is tested using that test data. While testing the system by using test data errors are again uncovered and corrected by using above testing steps and corrections are also noted for future use.

**Using Live Test Data:**

Live test data are those that are actually extracted from organization files. After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities. Then, the systems person uses this data as a way to partially test the system. In other instances, programmers or analysts extract a set of live data from the files and have them entered themselves.

It is difficult to obtain live data in sufficient amounts to conduct extensive testing. And, although it is realistic data that will show how the system will perform for the typical processing requirement, assuming that the live data entered are in fact typical, such data generally will not test all combinations or formats that can enter the

system. This bias toward typical values then does not provide a true systems test and in fact ignores the most likely to cause system failure.

### **Using Artificial Test Data:**

Artificial test data are created solely for test purposes, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information systems department, make possible the testing of all login and control paths through the program.

The most effective test programs use artificial test data generated by persons other than those who wrote the programs. Often, an independent team of testers formulates a testing plan.

### **User Training**

Whenever a new system is developed, user training is required to educate them about the working of the system so that it can be put to efficient use by those for whom the system has been primarily designed. For this purpose the normal working of the project was demonstrated to the prospective users. Its working is easily understandable and since the expected users are people who have good knowledge of computers, the use of this system is very easy.

### **Maintenance**

This covers a wide range of activities including correcting code and design errors. To reduce the need for maintenance in the long run, we have more accurately defined the user's requirements during the process of system development. Depending on the requirements, this system has been developed to satisfy the needs to the largest possible extent. With development in technology, it may be possible to add many more features based on the requirements in future. The coding and designing is simple and easy to understand which will make maintenance easier.

**Testing strategy**

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

**System Testing:**

Software once validated must be combined with other system elements (e.g. Hardware, people, database). System testing verifies that all the elements are proper and that overall system function performance is

achieved. It also tests to find discrepancies between the system and its original objective, current specifications and system documentation.

**Unit Testing:**

In unit testing different are modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goals to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be working satisfactorily as regards to the expected output from the module.

## CHAPTER 9

### CONCLUSION AND FUTUREWORK

In this paper, we present the HybridSeg framework which segments tweets into meaningful phrases called segments using both global and local context. Through our framework, we demonstrate that local linguistic features are more reliable than term-dependency in guiding the segmentation process. This finding opens opportunities for tools developed for formal text to be applied to tweets which are believed to be much more noisy than formal text. Tweet segmentation helps to preserve the semantic meaning of tweets, which subsequently benefits many downstream applications, e.g., named entity recognition. Through experiments, we show that segment-based named entity recognition methods achieves much better accuracy than the word-based alternative. We identify two directions for our future research. One is to further improve the segmentation quality by considering more local factors. The other is to explore the effectiveness of the segmentation-based representation for tasks like tweets summarization, search, hashtag recommendation, etc.

## CHAPTER 10

### REFERENCES

- [1] C. Li, J. Weng, Q. He, Y. Yao, A. Datta, A. Sun, and B.-S. Lee, “Twiner: Named entity recognition in targeted twitter stream,” in Proc. 35th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval, 2012, pp. 721–730.
- [2] C. Li, A. Sun, J. Weng, and Q. He, “Exploiting hybrid contexts for tweet segmentation,” in Proc. 36th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval, 2013, pp. 523–532.
- [3] A. Ritter, S. Clark, Mausam, and O. Etzioni, “Named entity recognition in tweets: An experimental study,” in Proc. Conf. Empirical Methods Natural Language Process., 2011, pp. 1524–1534.
- [4] X. Liu, S. Zhang, F. Wei, and M. Zhou, “Recognizing named entities in tweets,” in Proc. 49th Annu. Meeting Assoc. Comput. Linguistics: Human Language Technol., 2011, pp. 359–367.
- [5] X. Liu, X. Zhou, Z. Fu, F. Wei, and M. Zhou, “Extracting social events for tweets using a factor graph,” in Proc. AAAI Conf. Artif. Intell., 2012, pp. 1692–1698.
- [6] A. Cui, M. Zhang, Y. Liu, S. Ma, and K. Zhang, “Discover breaking events with popular hashtags in twitter,” in Proc. 21st ACM Int. Conf. Inf. Knowl. Manage., 2012, pp. 1794–1798.
- [7] A. Ritter, Mausam, O. Etzioni, and S. Clark, “Open domain event extraction from twitter,” in Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining, 2012, pp. 1104–1112.
- [8] X. Meng, F. Wei, X. Liu, M. Zhou, S. Li, and H. Wang, “Entitycentric topic-oriented opinion summarization in twitter,” in Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining, 2012, pp. 379–387.
- [9] Z. Luo, M. Osborne, and T. Wang, “Opinion retrieval in twitter,” in Proc. Int. AAAI Conf. Weblogs Social Media, 2012, pp. 507–510.
- [10] X. Wang, F. Wei, X. Liu, M. Zhou, and M. Zhang, “Topic sentiment analysis in twitter: a graph-based hashtag sentiment classification approach,” in Proc. 20th ACM Int. Conf. Inf. Knowl. Manage., 2011, pp. 1031–1040.